

Formulários, Fetch, Axios e APIs JSON

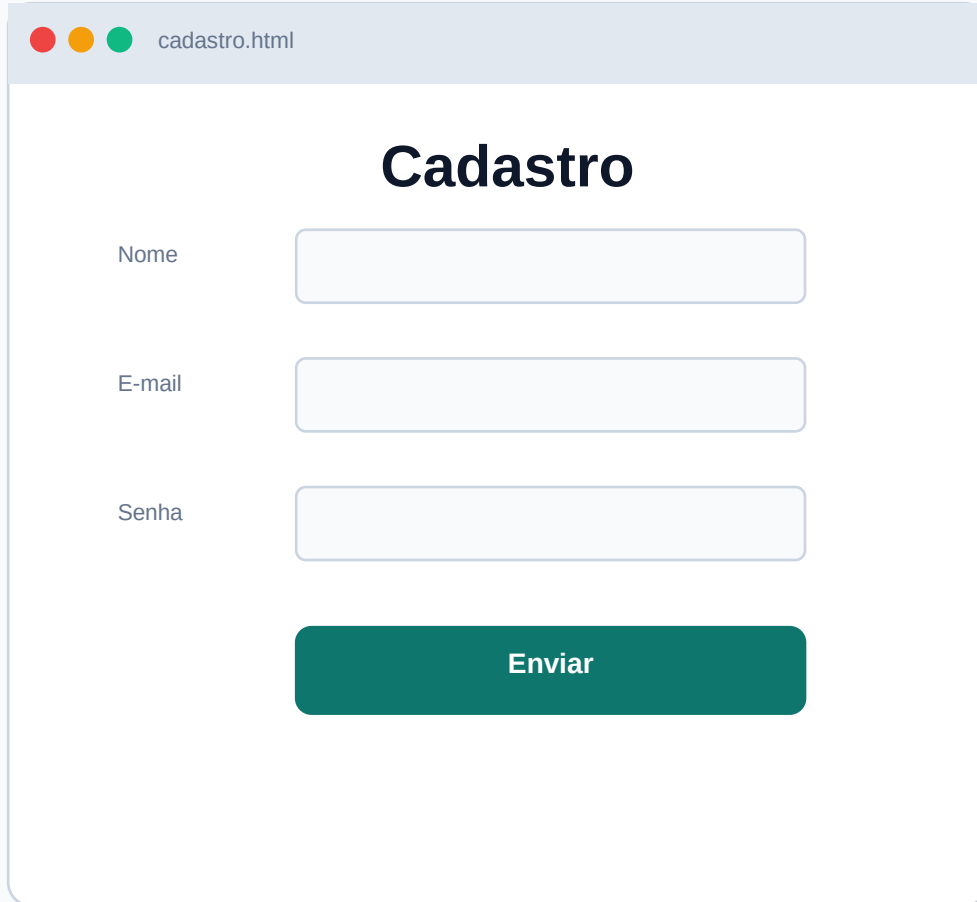
```
1 const titulo = document.querySelector("h1");
2 const lista = document.querySelectorAll(".item");
3 const card = document.createElement("div");
4 card.textContent = "Novo conteúdo";
5 main.appendChild(card);
```

Ideia central

O DOM é a ponte entre o JavaScript e o HTML. Tudo que o usuário vê, clica ou digita precisa ser encontrado, lido ou alterado pelo JS.

- Selecionar elementos
- Ler valores
- Criar elementos
- Alterar conteúdo
- Inserir ou remover da página

O que é um formulário?



The image shows a web browser window with the title bar 'cadastro.html'. The main content area has a title 'Cadastro' in bold. Below the title are three input fields, each with a label to its left: 'Nome', 'E-mail', and 'Senha'. At the bottom of the form is a green button with the text 'Enviar'.

- Um formulário agrupa campos de entrada.
- Cada campo representa um dado.
- O botão submit dispara o envio.
- Com JavaScript, nós controlamos esse envio.

Exemplos reais

Login, cadastro, busca, contato, comentário, avaliação, filtro de produtos, consulta de CEP e envio de tarefas.

```
1 <form id="formCadastro">
2   <label for="nome">Nome</label>
3   <input type="text" id="nome" name="nome">
4
5   <label for="email">E-mail</label>
6   <input type="email" id="email" name="email">
7
8   <button type="submit">Enviar</button>
9 </form>
```

<form>

Agrupar os campos e representar o formulário como um todo.

<label>

Texto descritivo ligado ao input. Melhora usabilidade e acessibilidade.

<input>

Campo onde o usuário digita ou escolhe uma informação.

<button type="submit">

Botão que aciona o evento de envio do formulário.

id, name e value: não é tudo a mesma coisa

Atributo	Para que serve	Exemplo de uso
id	Identifica um elemento na página	Usado pelo label e pelo JS
name	Identifica o dado enviado	Importante para FormData e back-end
value	Valor atual do campo	É o que o usuário digitou

```
1 <input id="nome" name="nome" value="Ariadne">
```

Principais tipos de input

type="text"

Texto comum: nome, cidade, busca.

type="email"

E-mail; o navegador pode validar formato.

type="password"

Campo oculto para senha.

type="number"

Números; pode ter min, max e step.

type="date"

Data com seletor do navegador.

type="checkbox"

Múltipla escolha: sim/não, interesses.

type="radio"

Uma opção dentro de um grupo.

type="submit"

Botão de envio do formulário.

Comportamento padrão do formulário

Por padrão, ao enviar um formulário, o navegador tenta mandar os dados e recarregar a página.

Clicar em Enviar

Submit

Navegador envia

Página recarrega

O problema em aulas de JS

Se a página recarrega, o console pode limpar, os elementos criados somem.

A solução

Controlar o envio com JavaScript usando o evento submit e **event.preventDefault()**.

```
1 form.addEventListener("submit", function(event) {  
2   event.preventDefault();  
3 });
```

Evento submit: o ponto de entrada

Em formulário, prefira ouvir o submit do form — não apenas o click do botão.

```
1 const form = document.querySelector("#formCadastro");
2
3 form.addEventListener("submit", function(event) {
4   event.preventDefault();
5   console.log("Formulário enviado pelo JS");
6 });
```

Por que submit?

Funciona quando o usuário clica no botão e também quando pressiona Enter dentro de um campo.

O que é event?

É o objeto que representa o evento ocorrido. Ele contém informações e métodos, como `preventDefault()`.

O que é `preventDefault()`?

Cancela a ação padrão do navegador, permitindo que o JavaScript controle o fluxo.

Capturando valores com .value

Depois de impedir o reload, podemos ler o que o usuário digitou.

```
1 <form id="formBusca">
2   <input type="text" id="nome" placeholder="Digite um
   nome">
3   <button type="submit">Buscar</button>
4 </form>
```

```
1 const form = document.querySelector("#formBusca");
2
3 form.addEventListener("submit", function(event) {
4   event.preventDefault();
5
6   const nome = document.querySelector("#nome").value;
7   console.log(nome);
8 });
```

Atenção

.value só faz sentido em elementos de formulário: input, textarea, select etc.

Usuário digita

input.value

JS usa o dado

Validação simples antes de buscar dados

Antes de chamar uma API, valide o mínimo para evitar chamadas desnecessárias.

```
1 form.addEventListener("submit", function(event) {  
2   event.preventDefault();  
3  
4   const nome = document.querySelector("#nome").value.trim();  
5  
6   if (nome === "") {  
7     resultado.textContent = "Digite um nome para buscar.";  
8     return;  
9   }  
10  
11   console.log("Buscar:", nome);  
12 });
```

trim()

Remove espaços no começo e no fim. Evita aceitar “ ” como texto válido.

return

Interrompe a função. Se o campo estiver vazio, o código não continua.

Feedback

O usuário precisa saber o que aconteceu. Não basta falhar silenciosamente.

Exibindo o resultado no DOM

```
1 <div id="resultado"></div>
```

```
1 const resultado = document.querySelector("#resultado");  
2  
3 resultado.innerHTML = "";  
4  
5 const card = document.createElement("div");  
6 card.textContent = "Usuário encontrado";  
7  
8 resultado.appendChild(card);
```



FormData: lendo vários campos de uma vez

```
1 <form id="formContato">
2   <input name="nome" type="text">
3   <input name="email" type="email">
4   <textarea name="mensagem"></textarea>
5   <button type="submit">Enviar</button>
6 </form>
```

```
1 const form = document.querySelector("#formContato");
2 form.addEventListener("submit", function(event) {
3   event.preventDefault();
4
5   const dados = new FormData(form);
6
7   const nome = dados.get("nome");
8   const email = dados.get("email");
9   const mensagem = dados.get("mensagem");
10 });
```

FormData precisa de name

O método `get("nome")` procura pelo atributo `name="nome"`, não pelo `id`.

Quando usar?

Cadastros, formulários de contato e envio com muitos campos.

Quando não precisa?

Em exemplos pequenos, capturar `.value` diretamente é mais didático.

Por que precisamos de código assíncrono?

ASSÍNCRONO

Clique

JS pede dados

Servidor responde

JS atualiza DOM

Sem assíncrono

A página ficaria travada esperando a resposta.

Com assíncrono

O navegador continua funcionando enquanto a resposta não chega.

Promise: uma promessa de resultado

ASSÍNCRONO

pending

aguardando

fulfilled

resolvida com sucesso

rejected

rejeitada com erro

```
1 const promessa = new Promise((resolve, reject) => {  
2   // depois de algum tempo...  
3   resolve("Dados recebidos");  
4   // ou reject("Erro na busca");  
5 });
```

Duas formas de lidar com Promise

ASSÍNCRONO

then/catch ou async/await

```
1 fetch(url)
2   .then(resposta => resposta.json())
3   .then(dados => console.log(dados))
4   .catch(erro => console.log(erro));
```

.then() / .catch()

```
1 async function buscar() {
2   try {
3     const resposta = await fetch(url);
4     const dados = await resposta.json();
5     console.log(dados);
6   } catch (erro) {
7     console.log(erro);
8   }
9 }
```

async / await

async e await: leitura passo a passo

ASSÍNCRONO

async marca uma função assíncrona; await espera a Promise resolver.

```
1 async function buscarDados() {  
2   console.log("Antes da busca");  
3  
4   const resposta = await fetch(url);  
5   const dados = await resposta.json();  
6  
7   console.log("Depois da busca");  
8   console.log(dados);  
9 }
```

1

A função começa a executar normalmente.

2

Await → espera a resposta chegar.

3

Depois, continua a executar a próxima linha.

4

O restante da página não fica travado.

Tratando erro com try/catch

ASSÍNCRONO

```
1 async function buscarDados() {  
2   try {  
3     const resposta = await fetch(url);  
4     const dados = await resposta.json();  
5     mostrarResultado(dados);  
6   } catch (erro) {  
7     console.error("Erro ao buscar dados:", erro);  
8     resultado.textContent = "Não foi possível buscar os dados.";  
9   }  
10 }
```

try

Parte que será tentada → requisição e o processamento dos dados.

catch

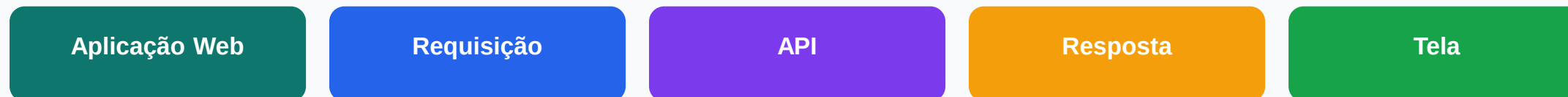
Parte executada se ocorrer erro: internet, URL, servidor, CORS etc.

Regra de ouro

Nunca deixe a tela “morrer em silêncio”. Mostre uma mensagem amigável ao usuário.

O que é uma API?

APIS



Exemplos de APIs

CEP, clima, mapas, pagamentos, login, redes sociais, produtos, notas, notícias e sistemas acadêmicos.

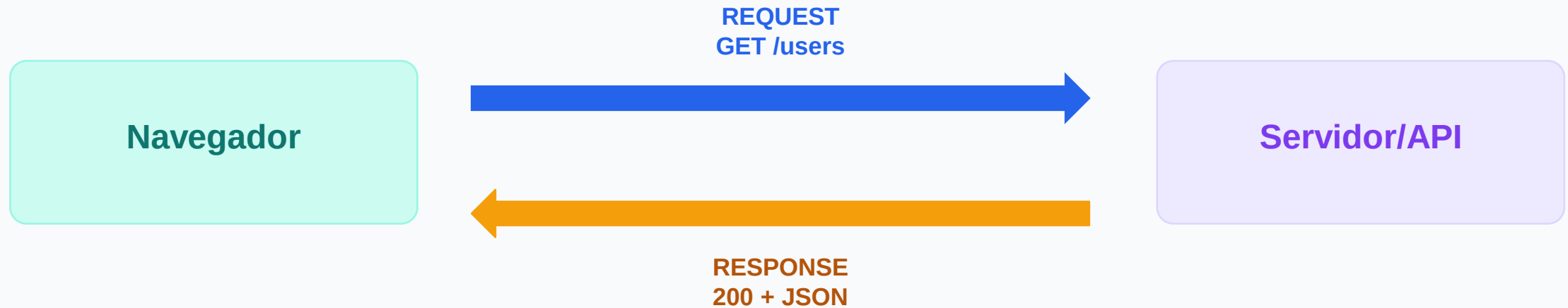
No nosso exemplo

A página vai pedir usuários a uma API e vai receber uma lista em formato JSON.

API é o serviço que entrega ou recebe dados.

Requisição e resposta HTTP

APIS



Request

Pedido: método, URL, parâmetros, cabeçalhos e, às vezes, corpo.

Response

Resposta: status, cabeçalhos e corpo com os dados.

Métodos HTTP mais usados

APIS

O método indica o tipo de ação que queremos fazer com os dados.

GET

buscar dados

POST

criar/enviar dados

PUT/PATCH

atualizar dados

DELETE

apagar dados

```
1 GET    /users    → listar usuários
2 GET    /users/1   → buscar usuário 1
3 POST   /users    → criar usuário
4 PUT    /users/1   → substituir usuário 1
5 PATCH  /users/1   → alterar parte do usuário 1
6 DELETE /users/1   → apagar usuário 1
```

Status HTTP: a resposta também fala

APIS

O status ajuda o JavaScript a saber se a requisição deu certo ou não.

200

OK

deu certo

201

Created

criado com sucesso

400

Bad Request

pedido inválido

404

Not Found

não encontrado

500

Server Error

erro no servidor

```
1 if (!resposta.ok) {  
2   throw new Error("Erro na requisição");  
3 }
```

Importante no fetch

O fetch só cai no catch por erro de rede. Para status 404 ou 500, normalmente precisamos verificar resposta.ok.

O que é JSON?

JSON é um formato de texto usado para trocar dados entre sistemas.

```
1 {  
2   "id": 1,  
3   "name": "Leanne Graham",  
4   "email": "leanne@example.com",  
5   "address": {  
6     "city": "Gwenborough"  
7   }  
8 }
```

- Parece objeto JavaScript, mas é texto.
- Usa pares chave: valor.
- Chaves geralmente ficam entre aspas duplas.
- Pode conter objetos e arrays.
- É o formato comum das respostas de APIs.

Frase simples

A API responde com texto JSON; o JavaScript transforma esse texto em dados que podemos usar.

JSON, array e objeto em JavaScript

A maioria das APIs responde com objeto, array ou array de objetos.

```
1 // Objeto
2 const usuario = {
3   id: 1,
4   name: "Leanne Graham",
5   email: "leanne@example.com"
6 };
7
8 console.log(usuario.name);
```

```
1 // Array de objetos
2 const usuarios = [
3   { id: 1, name: "Ana" },
4   { id: 2, name: "Bia" },
5   { id: 3, name: "Caio" }
6 ];
7
8 console.log(usuarios[0].name);
```

API

JSON

Array/Objeto

DOM

```
1 https://jsonplaceholder.typicode.com/users
```

```
1 [  
2   {  
3     "id": 1,  
4     "name": "Leanne Graham",  
5     "username": "Bret",  
6     "email": "Sincere@april.biz"  
7   }  
8 ]
```

Por que usar API de exemplo?

Permite treinar GET e POST sem risco de alterar dados reais.

Endpoint

É o endereço específico de um recurso dentro da API.

/users

Endpoint que retorna uma lista de usuários fictícios.

fetch: primeira requisição GET

FETCH

```
1 async function buscarUsuarios() {  
2   const resposta = await fetch(  
3     "https://jsonplaceholder.typicode.com/users"  
4   );  
5  
6   const usuarios = await resposta.json();  
7  
8   console.log(usuarios);  
9 }  
10  
11 buscarUsuarios();
```

1

fetch(url) envia a requisição.

2

await espera a resposta HTTP.

3

resposta.json() converte o corpo.

4

console.log mostra os dados.

fetch: por que precisa de .json()?

FETCH

A resposta HTTP vem em etapas: primeiro chega a resposta, depois lemos o corpo.

fetch(url)

Response

response.json()

Array de usuários

```
1 const resposta = await fetch(url);
2 console.log(resposta);           // objeto Response
3
4 const dados = await resposta.json();
5 console.log(dados);             // dados convertidos
```

Erro comum

Tentar usar resposta.name ou resposta.email. Esses dados não estão diretamente em resposta; eles estão no corpo convertido.

fetch + formulário: buscando pelo nome

FETCH

```
1 form.addEventListener("submit", async function(event) {  
2   event.preventDefault();  
3  
4   const nome = document.querySelector("#nome").value.trim();  
5  
6   const resposta = await fetch("https://jsonplaceholder.typicode.com/users");  
7   const usuarios = await resposta.json();  
8  
9   const filtrados = usuarios.filter(user =>  
10     user.name.toLowerCase().includes(nome.toLowerCase())  
11   );  
12  
13   mostrarResultado(filtrados);  
14 });
```

filter()

Percorre o array e devolve apenas os usuários que passam no teste.

includes()

Verifica se uma string contém outra.

toLowerCase()

Evita diferença entre maiúsculas e minúsculas.

Função para mostrar resultados

FETCH

```
1 function mostrarResultado(lista) {  
2   resultado.innerHTML = "";  
3  
4   if (lista.length === 0) {  
5     resultado.textContent = "Nenhum usuário encontrado.";  
6     return;  
7   }  
8  
9   lista.forEach(user => {  
10    const card = document.createElement("div");  
11    card.classList.add("card");  
12    card.innerHTML = `<strong>${user.name}</strong><br>  
13    ${user.email}`;  
14    resultado.appendChild(card);  
15  });  
16 }
```

Limpar antes

resultado.innerHTML = "" evita acumular resultados de buscas anteriores.

Caso vazio

Se não encontrar nada, mostramos uma mensagem e paramos com return.

Cards

Cada usuário vira um elemento criado dinamicamente no DOM.

fetch com tratamento de erro completo

FETCH

```
1 async function buscarUsuarios() {  
2   try {  
3     const resposta = await fetch(url);  
4  
5     if (!resposta.ok) {  
6       throw new Error("Erro HTTP: " + resposta.status);  
7     }  
8  
9     const usuarios = await resposta.json();  
10    mostrarResultado(usuarios);  
11  
12  } catch (erro) {  
13    console.error(erro);  
14    resultado.textContent = "Erro ao buscar usuários.";  
15  }  
16 }
```

try

fetch

ok?

json

catch
se
falhar

Por que verificar ok?

No fetch, uma resposta 404 ainda é uma resposta. Por isso, conferimos resposta.ok.

Mensagem ao usuário

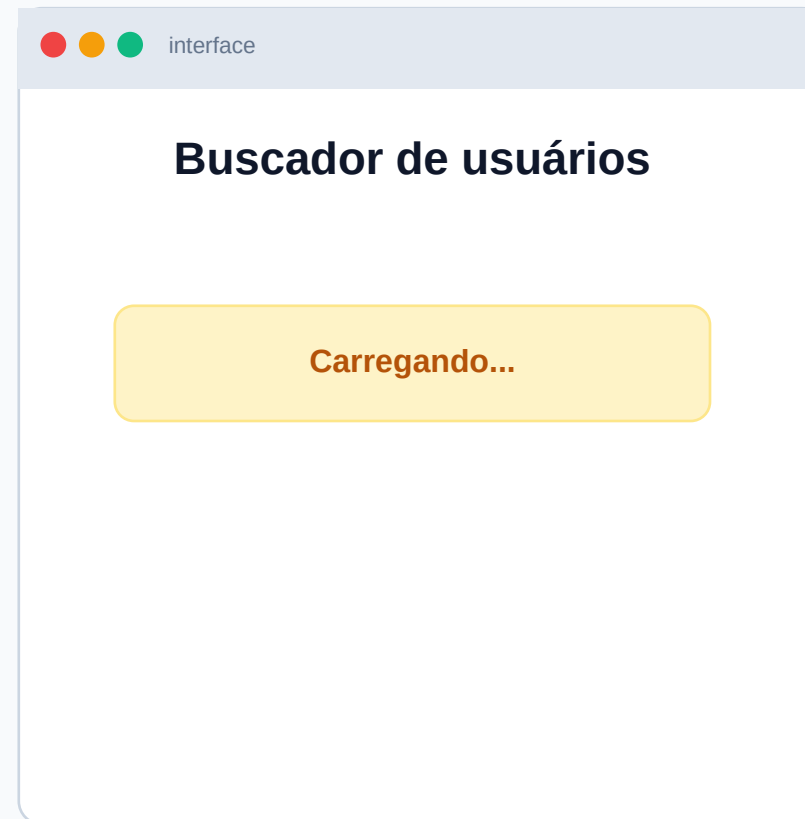
Erro no console é para o programador.
Texto na página é para o usuário.

Estado de carregamento: “Carregando...”

FETCH

Enquanto a resposta não chega, a interface deve informar que algo está acontecendo.

```
1 async function buscarUsuarios() {  
2   resultado.textContent = "Carregando...";  
3  
4   try {  
5     const resposta = await fetch(url);  
6     const usuarios = await resposta.json();  
7     mostrarResultado(usuarios);  
8   } catch (erro) {  
9     resultado.textContent = "Erro ao carregar dados.";  
10  }  
11 }
```



Axios: por que aprender?

AXIOS

Axios é uma biblioteca popular que simplifica requisições HTTP.

Fetch resolve

fetch é nativo e funciona muito bem.

```
1 // fetch
2 const resposta = await fetch(url);
3 const dados = await resposta.json();
```

Axios facilita

Axios reduz etapas, organiza respostas e melhora a experiência de uso em projetos maiores.

```
1 // axios
2 const resposta = await axios.get(url);
3 const dados = resposta.data;
```

A diferença central: com Axios, os dados já vêm em response.data.

```
1 <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

```
1 <!-- Ordem correta -->  
2 <script  
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js  
></script>  
3 <script src="script.js"></script>
```

```
1 <!-- Ordem problemática -->  
2 <script src="script.js"></script>  
3 <script  
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js  
></script>
```

Atenção à ordem

O Axios precisa ser carregado antes do seu script usar `axios.get()`.

Axios GET: primeira requisição

AXIOS

```
1 async function buscarUsuariosAxios() {  
2   const resposta = await axios.get(  
3     "https://jsonplaceholder.typicode.com/users"  
4   );  
5  
6   const usuarios = resposta.data;  
7  
8   console.log(usuarios);  
9 }  
10  
11 buscarUsuariosAxios();
```

1

axios.get(url) envia um GET.

2

await espera a resposta.

3

resposta.data contém os dados.

4

Não precisa chamar .json().

Axios + formulário: mesma lógica, menos conversão

AXIOS

```
1 form.addEventListener("submit", async function(event) {  
2   event.preventDefault();  
3  
4   const nome = document.querySelector("#nome").value.trim();  
5  
6   const resposta = await axios.get("https://jsonplaceholder.typicode.com/users");  
7   const usuarios = resposta.data;  
8  
9   const filtrados = usuarios.filter(user =>  
10     user.name.toLowerCase().includes(nome.toLowerCase())  
11   );  
12  
13   mostrarResultado(filtrados);  
14 });
```

Mesmo evento

submit + preventDefault continuam iguais.

Mesma filtragem

filter, includes e toLowerCase não mudam.

Muda a requisição

fetch + json() vira axios.get() + data.

```
1 async function buscarUsuarios() {  
2   try {  
3     const resposta = await axios.get(url);  
4     mostrarResultado(resposta.data);  
5   } catch (erro) {  
6     console.error(erro);  
7  
8     if (erro.response) {  
9       resultado.textContent = "Erro: " + erro.response.status;  
10    } else {  
11      resultado.textContent = "Erro de conexão.";  
12    }  
13  }  
14 }
```

erro.response

Existe quando o servidor respondeu com status de erro.

erro.message

Mensagem geral do erro.

catch

Lugar para registrar o erro e mostrar feedback ao usuário.

Atividade prática: Catálogo de séries com TVmaze

ATIVIDADE FINAL

Aplicação com formulário, requisição à API e cards criados dinamicamente no DOM.

Objetivo

Criar uma aplicação que permita pesquisar séries e programas de TV usando a API TVmaze.

Endpoint base

<https://api.tvmaze.com/search/shows?q=girls>
Depois, substituir girls pelo texto digitado pelo usuário.

Resultado esperado

Exibir os resultados em cards, um card para cada série encontrada.

Cada card deve apresentar

- Imagem/capa da série em formato JPG
- Nome da série
- Score retornado pela API
- Mensagem quando não houver imagem ou resultado

Requisitos técnicos

- Usar input e botão de busca
- Usar fetch ou Axios
- Converter/ler JSON
- Usar createElement/appendChild
- Limpar resultados antes de nova busca
- Tratar erro com try/catch

Desafio extra

Adicionar loading, validação para busca vazia e layout responsivo dos cards.